

Exercise:

Big Data Processing Architectures and Their Role in the Future of AI Systems

Introduction

Modern Artificial Intelligence (AI) systems are no longer isolated computational models operating on static datasets. Instead, they are embedded within complex, data-intensive architectures that continuously ingest, transform, index and serve data at scale. Large Language Models (LLMs), Retrieval-Augmented Generation (RAG) pipelines and autonomous AI agents fundamentally change how data systems are exercised: they amplify read workloads, require fresher data, introduce new data types such as embeddings and blur the line between analytical and operational processing.

This evolution did not occur in isolation. Over decades, data-intensive systems have progressed along several architectural dimensions: analytical processing engines optimized for large scans and aggregations; indexing structures designed to accelerate selective access; materialized views to precompute expensive queries; and, more recently, stream processing and event-driven systems to support continuous computation. Today, these components increasingly converge to support AI workloads that demand both scale and responsiveness.

This essay critically analyzes how classical analytical processing foundations, streaming systems and indexing paradigms interact and evolve under AI-driven workloads. It argues that AI systems push data architectures toward *incremental, continuously maintained analytical views with specialized serving layers*, rather than toward a single monolithic “unified” system. The future lies in principled composition: streaming ingestion, incremental computation and purpose-built indexes integrated through strong data lineage and consistency models.

1. Analytical Processing Foundations

1.1. Analytical vs. Transactional Processing

At the core of modern data architectures lies the distinction between **transactional processing (OLTP)** and **analytical processing (OLAP)**. Transactional systems are optimized for high-throughput, low-latency operations over small numbers of records, enforcing strong consistency and isolation. Their access patterns are dominated by point lookups, inserts and updates, often backed by B+ trees and row-oriented storage layouts.

Analytical systems, in contrast, are optimized for *read-heavy, scan-oriented workloads* that aggregate and summarize large volumes of historical data. Queries often touch

millions or billions of rows, performing joins, group-bys and complex expressions. To support this, analytical engines employ columnar storage, vectorized execution, cost-based optimizers and distributed execution across clusters.

The architectural separation between OLTP and OLAP reflects a fundamental trade-off: transactional systems prioritize correctness and latency per operation, while analytical systems prioritize throughput and efficiency over large datasets. Modern data warehouses - such as those built on distributed file systems or cloud object storage - extend classical OLAP ideas with elasticity, fault tolerance and decoupled storage and compute.

1.2. Indexes, Materialized Views and Query Optimization

Analytical performance depends not only on raw compute power but also on *precomputed structure*. Indexes, materialized views and query optimizers play complementary roles in reducing query cost.

Traditional index structures (e.g. B+ trees, bitmap indexes) accelerate selective predicates but are less effective for wide scans. As a result, analytical systems often rely more heavily on **materialized views**, which precompute joins and aggregations that would otherwise be prohibitively expensive. These views transform expensive runtime computation into cheaper storage reads.

Query optimizers orchestrate these components by selecting join orders, access paths and execution strategies. In distributed analytical systems, optimization also involves data placement, partition pruning and parallelism decisions. Importantly, modern optimizers increasingly reason about *incrementality*: whether a query result can be derived from previously computed state rather than recomputed from scratch.

1.3. Incremental View Maintenance at Scale

As data volumes and update rates grow, recomputing analytical views from scratch becomes infeasible. **Incremental View Maintenance (IVM)** addresses this challenge by updating query results as base data changes, rather than periodically rebuilding them.

IVM is critical at scale for several reasons. First, it reduces computational cost by transforming batch recomputation into smaller, continuous updates. Second, it lowers latency, allowing analytical results to reflect recent data changes. Third, it enables analytical systems to operate closer to real time, blurring the traditional boundary between batch analytics and operational reporting.

In distributed systems, incremental maintenance introduces complexity: updates must be propagated consistently, state must be partitioned and recovered and late or out-of-order data must be handled correctly. Nevertheless, incremental computation is increasingly foundational - not only for dashboards and metrics, but also for downstream systems such as feature stores and AI pipelines.

2. Streaming, Event Processing and CDC

2.1. Stream Processing vs. Incremental Analytics

Stream processing systems emerged to address the limitations of batch-oriented analytics in environments with continuous data arrival. Frameworks such as Apache Flink or Spark Structured Streaming enable developers to express computations over unbounded streams using windowing, stateful operators and event-time semantics.

Conceptually, **stream processing and incremental analytics address the same problem**: maintaining derived results as data changes. The difference lies primarily in abstraction and execution. Stream processors emphasize event-by-event processing with explicit state management, while analytical systems historically emphasized query-driven computation over static datasets.

In practice, these paradigms increasingly overlap. Modern streaming engines support SQL-like declarative queries, while analytical engines incorporate streaming ingestion and incremental refresh. This convergence reflects a broader architectural shift: from *periodic batch recomputation* toward *continuous, incremental maintenance of analytical state*.

2.2. Event Streaming Platforms and CDC Pipelines

Event streaming platforms such as Apache Kafka provide the backbone for data movement in modern architectures. They decouple producers from consumers, buffer high-throughput event streams and enable multiple downstream systems to react independently to the same data changes.

Change Data Capture (CDC) extends this idea by extracting fine-grained changes from transactional databases and publishing them as ordered event streams. CDC pipelines allow analytical systems, search indexes, caches and AI feature stores to remain synchronized with operational data without invasive polling or batch exports.

CDC plays a crucial role in keeping analytical views up to date. Instead of periodically dumping full tables, systems can propagate inserts, updates and deletes incrementally, enabling near-real-time analytics and reducing load on source systems. However, CDC introduces challenges around schema evolution, exactly-once delivery and consistency across multiple downstream consumers.

2.3. Latency, Consistency and Complexity Trade-offs

Streaming and CDC-based architectures trade simplicity for responsiveness. Lower latency often comes at the cost of increased system complexity, including state management, backpressure handling and failure recovery. Consistency guarantees may

also be weaker than in tightly coupled batch pipelines, especially when updates flow through multiple asynchronous components.

Architecturally, the key trade-off is not whether to use streaming, but *where incremental boundaries are drawn*. Systems must decide which computations are maintained continuously, which are recomputed periodically and how much inconsistency can be tolerated. These decisions become even more critical in AI-driven workloads, where freshness and correctness directly affect model behavior.

3. Implications for AI Systems

3.1. Incremental Computation as a First-Class Requirement

AI workloads fundamentally amplify the importance of incremental computation. LLM training pipelines ingest massive datasets that evolve over time, while fine-tuning and reinforcement learning workflows require continuous feedback loops. Reprocessing entire datasets for each update is prohibitively expensive.

In RAG systems, freshness is directly tied to output quality. New documents, user interactions, or corrections must be reflected quickly in retrieval indexes and metadata stores. Similarly, AI agents continuously generate new data - logs, actions, intermediate reasoning steps - that feed back into analytical and monitoring systems.

These patterns demand architectures that treat incremental updates as the norm rather than the exception. Incremental feature computation, continuously updated embeddings and rolling aggregations become foundational building blocks.

3.2. Read Amplification and Serving Pressure

AI systems dramatically increase **read amplification**. A single user query may trigger dozens or hundreds of retrievals, ranking operations and metadata lookups. Unlike traditional dashboards, which serve a limited number of human users, AI-powered applications operate at machine scale.

This read pressure shifts architectural emphasis toward **serving layers** optimized for low-latency access. Analytical warehouses alone are insufficient; their scan-oriented execution models are ill-suited for high-QPS, low-latency workloads. As a result, precomputed views, caches and specialized indexes become essential intermediaries between raw analytical data and AI inference systems.

3.3. Data Lineage, Freshness and Heterogeneity

AI systems increasingly combine structured data (tables), semi-structured data (JSON, logs), unstructured text and embeddings. Maintaining lineage across these representations is non-trivial. A single source update may require coordinated changes across relational views, vector indexes and derived features.

Freshness requirements further complicate system design. While some AI tasks tolerate eventual consistency, others - such as compliance checks or real-time decision-making - require stronger guarantees. Architectures must therefore balance performance with traceability, ensuring that model outputs can be explained and audited based on the underlying data state.

4. Vector Databases and New Indexing Paradigms

4.1. Comparison with Traditional Indexes and Views

Vector databases and embedding indexes represent a significant departure from traditional indexing techniques. Instead of exact-match or range predicates, they support **approximate nearest neighbor (ANN)** search in high-dimensional spaces. This fundamentally changes query semantics: results are ranked by similarity rather than by precise predicates.

From an architectural perspective, vector indexes resemble *materialized views over learned representations*. They precompute and store embeddings derived from raw data, enabling fast similarity search at inference time. However, unlike classical views, embeddings are probabilistic and often require periodic recomputation as models evolve.

4.2. Analytical System or Serving Layer?

Vector databases occupy an ambiguous position. On one hand, they store large datasets and support complex queries; on the other, they are optimized for low-latency serving rather than analytical exploration. In practice, they function best as **specialized serving layers** tightly integrated with upstream analytical systems.

Treating vector stores as standalone analytical systems risks duplication of ingestion, governance and consistency logic. Instead, they should be derived from authoritative data sources - such as warehouses or lakes - using streaming or CDC pipelines to propagate updates and trigger embedding recomputation.

4.3. Integration Challenges and Open Problems

Integrating vector databases into broader data architectures raises several challenges. Incremental updates are non-trivial, especially when embeddings depend on global model state. Consistency between relational metadata and vector representations must be maintained. Hybrid queries that combine structured filters with similarity search remain an active area of research and engineering.

These challenges highlight that vector databases are not a replacement for analytical systems, but an extension of the indexing landscape - one that must be carefully composed with existing components.

5. Technical Positioning and Future Outlook

5.1. Unified Architectures vs. Specialization

AI workloads do not eliminate the need for specialized systems; instead, they intensify it. While there is value in unifying interfaces and abstractions, attempting to collapse analytical processing, streaming computation and low-latency serving into a single system risks suboptimal performance and unmanageable complexity.

The more sustainable direction is **architectural unification at the conceptual level**, combined with **physical specialization**. Streaming systems handle ingestion and change propagation, analytical engines maintain authoritative state and complex views and serving layers - including vector indexes - optimize for inference-time access.

5.2. Central Components of Future Architectures

Streaming and incremental view maintenance will become increasingly central. Rather than treating streams as peripheral, future architectures will place them at the core, enabling continuous synchronization between operational data, analytical views and AI-serving layers.

Equally important are metadata management, lineage tracking and consistency models. As AI systems become decision-makers rather than passive analyzers, the ability to reason about data provenance and correctness becomes a first-order concern.

5.3. Architectural Principles for the Next Decade

Over the next 5–10 years, several principles will dominate system design:

- **Incrementality by default:** systems should assume continuous updates rather than periodic recomputation.
- **Separation of concerns:** analytical computation, streaming propagation and serving should be decoupled but well-integrated.
- **Data-centric AI:** architectures must treat data quality, freshness and lineage as core system properties.
- **Composable abstractions:** flexibility will come from composing specialized systems through well-defined interfaces, not from monolithic designs.

6. Conclusion

The evolution of AI workloads exposes both the strengths and limitations of existing data processing architectures. Analytical engines, streaming systems and indexing

techniques are not being replaced; they are being recontextualized within more dynamic, incremental and AI-driven pipelines.

The future of AI systems depends not on abandoning classical data architecture principles, but on extending them. Incremental computation, materialized views and specialized indexes - now including embeddings - form the backbone of scalable, responsive AI applications. By embracing principled composition rather than forced unification, data architectures can evolve to meet the demands of increasingly autonomous and data-hungry AI systems.

References (Indicative)

- Stonebraker et al., *The Case for Shared Nothing*, IEEE Computer
- Abadi et al., *The Design of the Postgres Storage System*
- Zaharia et al., *Discretized Streams: Fault-Tolerant Streaming Computation*
- Kleppmann, *Designing Data-Intensive Applications*
- Apache Flink, Kafka and Spark documentation
- Recent industry papers on RAG systems and vector search architectures